

BreachPilot

Research Document

MCP-Powered Security Triage Agent for Splunk

Project	BreachPilot: MCP-Powered Incident Commander for Splunk
Track	Security
Target bonus prize	Best Use of Splunk MCP Server
Live demo	https://breach-pilot.vercel.app
Repository	https://github.com/BeBecpp/BreachPilot
Document date	2026-06-14

Document purpose

This document records the research basis, problem framing, technical rationale, architecture, demo methodology, and evaluation strategy for BreachPilot. It is intentionally formatted as a plain research/report document rather than a visual pitch deck.

Important scope note: the public Vercel deployment demonstrates the workflow using synthetic Splunk-style data. The repository also defines a real Splunk MCP integration path through a client adapter and environment configuration. This distinction is explicit to keep the submission accurate and testable.

1. Executive Summary

BreachPilot is a security operations research prototype that applies agentic investigation workflows to Splunk data. The project addresses a common SOC pain point: after a suspicious alert, an analyst often has to pivot manually across authentication logs, VPN events, admin activity, data access logs, endpoint indicators, and network evidence. BreachPilot compresses that workflow into an explainable incident case file: timeline, evidence, risk score, and response actions.

The project is aligned with the Splunk Agentic Ops Hackathon Security track because the track asks for solutions that help security teams detect threats faster, investigate incidents more efficiently, and automate security workflows using AI and Splunk data [1]. It also maps directly to the Best Use of Splunk MCP Server bonus prize, which emphasizes intelligent, agent-driven experiences that connect AI agents to Splunk data for automated investigation, contextual insights, and real-time decision making [2].

The central research position is that a strong security AI prototype should not behave as an opaque chatbot. It should be evidence-first, query-transparent, and human-in-the-loop. The Splunk hackathon announcement also emphasizes that strong projects should keep humans in the loop and provide clearer next moves rather than automation for its own sake [3]. BreachPilot follows that principle by showing each investigation step and preserving the SPL-style query context that produced the result.

Research question	BreachPilot answer
How can AI reduce SOC triage time without hiding evidence?	Use an agent workflow to collect scoped evidence, then present timeline, risk score, and response actions with query transparency.
How can MCP be used meaningfully rather than as a buzzword?	Treat MCP as the investigation bridge between an AI agent and Splunk data/tooling, not as a generic chat layer.
How can a hackathon demo stay reliable without a live Splunk dependency?	Use demo mode for public Vercel testing and document a real Splunk MCP integration path for local/enterprise deployment.

2. Hackathon and Technology Context

The Splunk Agentic Ops Hackathon invites developers and security practitioners to create AI-powered solutions across Observability, Security, and Platform tracks. The Security track specifically focuses on faster threat detection, more efficient incident investigation, and automated security workflows using AI and Splunk data [1]. The project submission requirements also call for a functioning project, a public repository, clear README/setup instructions, dependencies, example configurations or datasets, and a root-level architecture diagram [1].

BreachPilot is intentionally positioned for the Security track and the MCP Server bonus category. The bonus category is defined around AI agents connected to Splunk data, with workflows such as automated investigation, contextual insights, and real-time decision making [2]. Therefore, the prototype emphasizes an investigation workflow rather than a general-purpose assistant.

Requirement / judging signal	Design implication for BreachPilot
Security track: investigate incidents efficiently	The demo scenario is an account takeover investigation that correlates auth, VPN, admin, data-access, endpoint, and network events.
MCP Server bonus: agent-driven experiences	The UI shows an investigation plan, MCP-style tool calls, SPL-style queries, and evidence retrieval steps.
Public repository with runnable project	The repo includes a Vercel demo, Python local backend, sample dataset, README, and architecture diagram.
Video under 3 minutes	The research narrative is condensed into a simple demo flow: problem, entity input, workflow, evidence, value.

3. Research Problem: SOC Triage Bottleneck

Security operations centers handle alerts that are often incomplete. A suspicious login may require several follow-up questions: was there brute-force activity before the login, did VPN behavior change, did the user access privileged systems, did the user request sensitive exports, and did related hosts show service anomalies? Each question often becomes a separate query and mental context switch.

This problem is not that analysts lack data. The problem is that the data is distributed across sourcetypes, indexes, and event formats. An assistant that only summarizes logs after a single search does not solve the triage bottleneck. The more useful model is an agent that performs a constrained sequence of evidence-collection steps, then makes the result auditable.

The demo scenario is mapped to common adversary behavior categories. MITRE ATT&CK documents Valid Accounts (T1078) as abuse of existing credentials for access, persistence, privilege escalation, or defense evasion [8]. Brute Force (T1110) describes repeated credential guessing activity [9]. Exfiltration Over C2 Channel (T1041) covers theft of data over an existing command-and-control communication path [10]. BreachPilot does not claim to fully detect those techniques; instead, it uses them as realistic framing for the event chain and analyst questions.

4. Technical Rationale: Why Splunk MCP Server

Splunk MCP Server is relevant because it provides a standardized and secure interface for AI assistants and agents to interact with Splunk Enterprise or Splunk Cloud. Splunkbase describes MCP as a universal adapter for AI applications and external data sources, and the Splunk MCP Server as a way to connect intelligent systems with Splunk platform data [4].

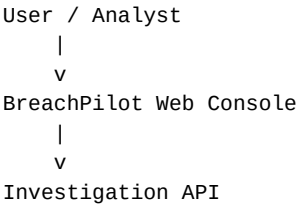
The MCP Server tooling also contains important guardrails. Splunk documentation states that `run_splunk_query` is intended for quick searches that are safe and non-destructive. The tool may fail if a search is unsafe or destructive, if execution exceeds 1 minute, or if the response exceeds 1000 events [5]. This strongly influences BreachPilot design: the agent should execute narrow, scoped searches rather than broad exploratory queries.

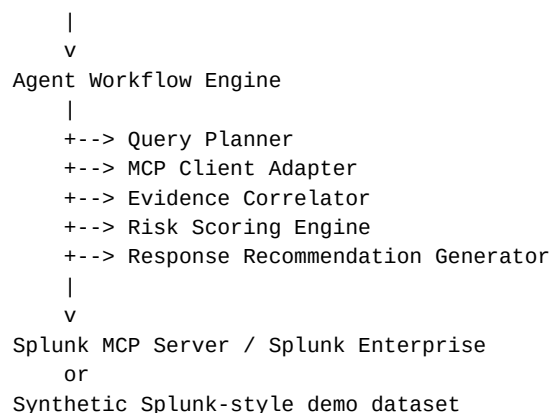
This is why the prototype favors small, explainable SPL-style searches. The agent does not attempt to pull entire indexes or perform destructive actions. It uses entity-focused searches, correlates evidence, and recommends response actions for a human analyst to review.

MCP design constraint	BreachPilot implementation choice
Safe and non-destructive queries	Only read-oriented investigation queries are shown and used.
Short execution time	Queries are scoped around one entity, IP, host, or recent incident window.
Limited event response size	The workflow collects only the fields needed for timeline and evidence cards.
Human verification	The UI shows the workflow steps and query text rather than hiding the reasoning path.

5. System Architecture

BreachPilot uses a layered design that separates the user interface, investigation API, agent workflow, MCP adapter, correlation logic, and scoring logic. This makes the Vercel demo reliable while preserving a real Splunk MCP integration path.





Component	Responsibility
Web Console	Accepts an entity and displays the risk score, timeline, evidence cards, actions, and SPL/MCP transparency.
Investigation API	Receives the entity request and returns the normalized incident case file.
Agent Workflow Engine	Plans the sequence of investigation steps and controls the evidence collection process.
MCP Client Adapter	Abstracts the interaction with real Splunk MCP Server or demo-mode event retrieval.
Evidence Correlator	Groups events across sourcetypes into coherent evidence categories.
Risk Scoring Engine	Assigns a severity score based on weighted evidence signals.

6. Investigation Workflow Design

The core workflow is designed as a deterministic investigation plan rather than an unconstrained autonomous agent. This reduces risk, improves repeatability, and makes the demo easy to evaluate.

Step	Agent action	Analyst-visible output
1	Normalize the input entity as user, IP address, or host.	Entity banner and investigation mode.
2	Run scoped Splunk-style search for recent events.	MCP/SPL query panel and raw event count.
3	Correlate events across authentication, VPN, admin, data access, endpoint, and network sourcetypes.	Attack path timeline.
4	Extract risk signals such as repeated failures, success from suspicious IP, export request, and suspicious egress.	Evidence cards grouped by category.
5	Score severity and confidence.	Risk score and incident summary.
6	Generate recommended response actions.	Human-in-the-loop next actions.

Representative SPL-style queries used by the investigation are shown below. These are intentionally narrow and entity-focused.

```

index=breachpilot user="jane.admin" earliest=-24h
| table _time sourcetype user src_ip host action status risk_signal
| sort _time

index=breachpilot user="jane.admin" action IN ("failed_login", "successful_login")
| stats count by action src_ip

index=breachpilot (user="jane.admin" OR src_ip="45.77.12.8")
| sort _time
| table _time sourcetype user src_ip host action status risk_score risk_signal

```

7. Demo Dataset and Scenario

The current Vercel deployment uses a synthetic dataset. This is acceptable for a research prototype because it makes the demo reproducible and avoids exposing private security data. The dataset is shaped like Splunk events: timestamp, sourcetype, user, source IP, host, action, status, risk signal, and risk score.

Primary demo entity: jane.admin. The incident narrative is a suspected account takeover followed by suspicious data access. The sequence includes multiple failed logins, a successful login from the same suspicious IP, unusual VPN activity, admin console access, a sensitive export request, suspicious egress, and a service error spike.

Event class	Evidence signal	Analyst interpretation
Authentication	Multiple failed logins before success from same IP.	Possible brute-force or credential stuffing behavior.
VPN	New or unusual remote-access session.	Potential post-authentication access path.
Admin activity	Admin console accessed after suspicious login.	Higher-impact account activity.
Data access	Customer export endpoint requested.	Potential data staging or exfiltration preparation.
Network	Suspicious outbound domain contacted.	Potential command-and-control or exfiltration path.
Application	checkout-api-01 error spike.	Possible related operational impact or follow-up pivot.

8. Risk Scoring and Evidence Model

BreachPilot uses an interpretable scoring model rather than a black-box model. This is deliberate: a hackathon SOC tool should demonstrate analyst utility, not just output a number. Each risk feature contributes to the final severity and is also shown as evidence.

Feature	Weight rationale
Failed login cluster	Raises suspicion but is not sufficient alone.
Success after failure from same IP	Strong account-takeover indicator.
Unusual VPN or first-seen IP	Adds context about anomalous access location or infrastructure.
Admin console access	Increases impact because the account has privileged activity.
Sensitive export request	Indicates possible data access or staging.
Suspicious egress	Indicates possible external communication after compromise.

The model is intentionally adjustable. In a production system, weights would be calibrated against environment-specific baselines, known-good behavior, and historical incidents. The current implementation focuses on explainability for the demo.

9. Implementation and Deployment Status

The project has two execution paths: public demo mode and real integration mode. Public demo mode powers the Vercel deployment at <https://breach-pilot.vercel.app>. Real integration mode is represented by the repository structure and MCP client adapter, which would send scoped searches through Splunk MCP Server in an environment with Splunk credentials and MCP configuration.

Mode	Purpose	Status
Demo mode	Reliable public demo for judges and video recording using synthetic data.	Implemented and deployed on Vercel.
Python local mode	Local reproducible backend for testing the investigation workflow.	Included in repository.
Real Splunk MCP mode	Connect the same workflow to Splunk Enterprise or Splunk Cloud through	Integration path defined; requires local Splunk/MCP credentials.

	MCP.	
--	------	--

This split is a practical hackathon tradeoff. It ensures the evaluator can open a working product immediately while the architecture still demonstrates where Splunk MCP fits in a real deployment.

10. Evaluation Plan

BreachPilot can be evaluated through both functional and qualitative criteria. Functional evaluation checks whether the app consistently converts the demo entity into the expected timeline, evidence, and recommendations. Qualitative evaluation checks whether the workflow is understandable to a security analyst and whether the MCP/SPL transparency supports trust.

Evaluation area	Metric or check
Correctness	The jane.admin scenario should produce the expected account-takeover case file.
Transparency	The UI should show investigation steps and SPL-style query text.
Usability	A judge should understand the value in under 60 seconds.
MCP alignment	The design should use MCP as a controlled data/tool access layer, not just as a label.
Safety	No destructive actions; recommendations remain human-in-the-loop.
Reproducibility	The public demo and sample dataset should run without private credentials.

11. Limitations and Future Work

The current prototype is designed for a hackathon demonstration, not production SOC deployment. Its most important limitations are:

- The public demo uses synthetic data rather than a live Splunk index.
- The real Splunk MCP path requires a configured Splunk instance, MCP Server, token-based authentication, and environment-specific indexes.
- The risk scoring model is interpretable but manually weighted; production use would require calibration and validation.
- The workflow recommends containment actions but does not execute them automatically, by design.
- The demo scenario focuses on account takeover and possible data access; broader coverage would require more scenario packs and detections.

Future work should add live MCP Server authentication, saved-search support, analyst feedback capture, ATT&CK technique mapping in the UI, exportable incident reports, and integration with Splunk Enterprise Security/SOAR workflows. A production-grade version should also include role-based access control, audit logging for agent actions, configurable query limits, and environment-specific data model mapping.

12. Conclusion

BreachPilot demonstrates a research-backed approach to agentic security operations: AI should help analysts ask the next right question, collect evidence, and produce an auditable case file. The project is strongest when framed as an MCP-powered investigation workflow rather than a chatbot. It aligns with the Splunk Agentic Ops Hackathon by using operational security data as the context for AI, keeping the human analyst in control, and showing how MCP can orchestrate meaningful evidence collection across Splunk data.

References

- [1] Splunk Agentic Ops Hackathon Official Rules, Devpost. Security track and submission requirements. <https://splunk.devpost.com/rules>
- [2] Splunk Agentic Ops Hackathon Official Rules, Devpost. Best Use of Splunk MCP Server bonus prize description. <https://splunk.devpost.com/rules>
- [3] Splunk Blog, "Announcing the Splunk Agentic Ops Hackathon". https://www.splunk.com/en_us/blog/artificial-intelligence/splunk-agentic-ops-hackathon.html
- [4] Splunkbase, "Splunk MCP Server" app listing. <https://splunkbase.splunk.com/app/7931>
- [5] Splunk Documentation, "MCP Server tools" and guardrails for run_splunk_query. <https://help.splunk.com/en/splunk-cloud-platform/mcp-server-for-splunk-platform/1.1/mcp-server-tools>
- [6] Splunk Documentation, "Splunk AI Assistant for SPL" overview. <https://docs.splunk.com/Documentation/AIAssistant/latest/User/AboutAIAssistant>
- [7] Splunkbase, "Splunk AI Toolkit" app listing. <https://splunkbase.splunk.com/app/2890>
- [8] MITRE ATT&CK, "Valid Accounts (T1078)". <https://attack.mitre.org/techniques/T1078/>
- [9] MITRE ATT&CK, "Brute Force (T1110)". <https://attack.mitre.org/techniques/T1110/>
- [10] MITRE ATT&CK, "Exfiltration Over C2 Channel (T1041)". <https://attack.mitre.org/techniques/T1041/>
- [11] Splunk MCP Server GitHub README. <https://github.com/splunk/splunk-mcp-server2/blob/main/README.md>

Appendix A: Suggested Video Evidence Checklist

1. Open the live demo URL: <https://breach-pilot.vercel.app>
2. Enter jane.admin and start the investigation.
3. Show the risk score and severity classification.
4. Show the attack timeline and evidence cards.
5. Show the recommended analyst response actions.
6. Scroll to the MCP/SPL workflow transparency section.
7. State clearly that the public deployment uses demo mode and the repository documents the real Splunk MCP integration path.